METHODOLOGY

A foundational understanding of mathematical optimization, associated solution methodologies, and graph theory is essential. The preceding chapters outlined the underlying problem, the rationale for this study, the applicability of the multiple target access problem to graph networks, and the use of Integer Programming as an exact solution method.

The initial model proposed in this research was a mixed-integer linear programming (MILP) formulation employing a single-commodity flow approach. However, this formulation exhibited a significant limitation: it strictly constrained target nodes to act as static terminal nodes (leaves), thereby preventing optimal paths from traversing through them to reach subsequent targets.

To resolve this limitation, significantly reduce computational time, and allow target nodes to function flexibly as intermediate “pass-through” nodes, the problem has been reformulated utilizing a **Multi-Commodity Flow Formulation** applied to the Steiner Arborescence problem.

This section details the mathematical formulations of both the initial and the revised models. It examines the structural limitations that previously forced target nodes to act as static terminal leaves, and demonstrates how the multi-commodity formulation enables flexible, pass-through targets.

1. The Initial Single-Commodity Flow Model

The initial model used a single-commodity flow formulation. To represent flow on the network, it utilized integer flow variables along with binary decision variables and dummy variables. The model variables and parameters are defined as follows:

Model variables and parameters:

d_i = Binary decision variable; 1 if edge i is active in the optimal route, 0 otherwise

f_i = Integer flow variable on edge i (value in $[0, n_T]$)

$u_{j,i}$ = Binary dummy variable for target point j on edge i

I_i = The impedance (traversing cost) of traveling along edge i

T = Set of target nodes ($T \subset V$)

V = Set of all vertices in the network

E = Set of all directed edges in the network

$start$ = The starting vertex (source/root of the arborescence)

n_T = Total number of target vertices

Model equations:

The objective was to minimize the total impedance of the active edges:

$$\text{Minimize: } Z = \sum_{i \in E} d_i \times I_i$$

Subject to the following constraints:

Source Outflow:

$$\sum_{i \in outgoing(start)} f_i = n_T$$

Intermediate Node Constraint:

$$\sum_{i \in incoming(v)} f_i - \sum_{j \in outgoing(v)} f_j = 0 \quad \forall v \in V - (T \cup \{start\})$$

Dummy Link 1:

$$f_i = d_i + \sum_{j=1}^{n_T - 1} u_{j,i} \quad \forall i \in E$$

Dummy Link 2:

$$d_i \geq u_{j,i} \quad \forall i \in E, \forall j \in \{1, \dots, n_T - 1\}$$

Target Node Inflow:

$$\sum_{i \in incoming(v)} f_i = 1 \quad \forall v \in T$$

$$d_i \in \{0,1\} \quad \forall i \in E$$

$$f_i \in \mathbb{Z}^+ \cup \{0\} \quad \forall i \in E$$

$$u_{j,i} \in \{0,1\} \quad \forall j \in \{1, \dots, n_T - 1\}, \forall i \in E$$

The Limitation of the Previous Model (Static Terminal Nodes)

In the previous model, target nodes were strictly treated as terminal nodes (leaves). This was caused by two modeling decisions: 1. **Exclusion from Flow Conservation:** Target nodes T were

excluded from the intermediate nodes list ($V - (T \cup \{start\})$), meaning they did not have flow conservation constraints. 2. **Fixed Inflow Constraint:** The inflow to each target node was constrained to be exactly 1:

$$\sum_{i \in incoming(v)} f_i = 1 \quad \forall v \in T$$

Because of this, if an optimal path to target B necessitated traversing target A, the flow entering target A would require a minimum value of 2 (1 unit terminating at target A, and at least 1 unit flowing through to target B). However, the target node inflow constraint restricted the incoming flow to target A to exactly 1, creating a mathematical contradiction. Consequently, target nodes could never function as intermediate nodes and were forced to remain terminal leaves (as illustrated in Figure 7). This rigidity resulted in circuitous, suboptimal routing trees.

---

2. The New Multi-Commodity Flow Model

To resolve the terminal node limitation and optimize solving time, the model has been reformulated using a Multi-Commodity Flow approach. A separate commodity k is defined for each target node T_k. The model variables and parameters are defined as follows:

Model variables and parameters:

D_i = Binary decision variable; 1 if edge i is active in the optimal route, 0 otherwise

$y_{k,i}$ = Continuous flow variable of commodity k on edge i (value in $[0,1]$)

I_i = The impedance (traversing cost) of traveling along edge i

T = Set of target nodes ($T \subset V$)

V = Set of all vertices in the network

E = Set of all directed edges in the network

$start$ = The starting vertex (source/root of the arborescence)

n_T = Total number of target vertices

Model equations:

Given a directed graph $G = (V, E)$, the objective is to minimize the total impedance of the activated edges:

$$\text{Minimize: } Z = \sum_{i \in E} D_i \times I_i$$

Subject to the following constraints:

Capacity Link (C1):

$$y_{k,i} \leq D_i \quad \forall k \in \{1, \dots, n_T\}, \forall i \in E$$

Source Outflow (C2):

$$\sum_{i \in outgoing(start)} y_{k,i} = 1 \quad \forall k \in \{1, \dots, n_T\}$$

Node Balance (C3):

$$\sum_{i \in incoming(v)} y_{k,i} - \sum_{j \in outgoing(v)} y_{k,j} = \text{demand}(k, v) \quad \forall v \in V - \{start\}, \forall k \in \{1, \dots, n_T\}$$

where the demand for commodity k at node v is defined as:

$$\text{demand}(k, v) = \begin{cases} 1 & \text{if } v = T_k \\ 0 & \text{otherwise} \end{cases}$$

$$D_i \in \{0,1\} \quad \forall i \in E \quad \text{(C4)}$$

$$y_{k,i} \in [0,1] \quad \forall k \in \{1, \dots, n_T\}, \forall i \in E \quad \text{(C5)}$$

$$I_i \in \mathbb{R}^+ \quad \forall i \in E \quad \text{(C6)}$$

Detailed Interpretation of the Constraints

Constraint (C1) - Capacity Link: This constraint links the continuous commodity flows $y_{k,i}$ to the binary edge activation variables D_i. If any commodity k traverses edge i (so $y_{k,i} > 0$), then D_i must be set to 1. Since the objective function minimizes the sum of active edge costs, D_i will be 0 if no commodity uses edge i. This represents a tight linear relaxation that allows solvers to solve the problem in milliseconds without exploring a large branch-and-bound tree.

Constraint (C2) - Source Outflow: Enforces that the start node (root) must exit exactly 1 unit of flow for each commodity k. This initiates the path for each target.

Constraint (C3) - Node Balance (Flow Conservation): This is the core of the multi-commodity formulation. For each commodity k: - If v is the designated target for commodity k ($v = T_k$), the net flow entering v must be exactly 1. - If v is any other node (intermediate or a target for a different commodity), the net flow entering must equal the net flow leaving (demand = 0).

---

3. Summary of Target Flexibility and Performance Gains

How the New Model Enables Flexible, Pass-Through Targets

The new multi-commodity flow formulation naturally resolves the terminal node limitation:

1. Commodity-Specific Sinks: A target node v behaves as a sink (demand = 1) *only* for its own commodity flow $y_{v,i}$.

2. Pass-Through Flexibility: For all other commodities $m \neq v$, the target node v behaves exactly like an intermediate node (demand = 0), allowing other commodity flows to enter and exit freely:

$$\sum_{i \in incoming(v)} y_{m,i} = \sum_{j \in outgoing(v)} y_{m,j}$$

3. True Optimal Paths: This mathematical formulation permits target nodes to serve as pass-through (intermediate) nodes for other paths (as illustrated in Figure 8), allowing the model to ascertain the true optimal Steiner Tree arborescence.

4. Reduced Complexity: Additionally, by representing commodity flows as continuous variables on $[0,1]$ and linking them via $y_{k,i} \leq D_i$, the formulation eliminates the necessity for the complex binary dummy variables $u_{j,i}$ utilized in the previous model. This vastly reduces the number of constraints and variables, thereby significantly improving computational efficiency.

Solver Performance Comparison

Initial Single-Commodity Model: Single-commodity flow formulations inherently possess weak linear programming (LP) relaxations when employing large capacity constraints (e.g., $f_i \leq K \cdot$

d_i). Furthermore, this model relied on complex binary dummy variables ($u_{j,i}$) to maintain sub-tour elimination. This bloated the matrix size exponentially, causing the GLPK solver to stall and explore millions of branch-and-bound nodes without effectively closing the optimality gap.

New Multi-Commodity Model: By removing the binary dummy variables and replacing them with continuous commodity variables bounded on $[0,1]$, the matrix density decreases significantly. More importantly, the formulation features an extremely tight LP relaxation ($y_{k,i} \leq D_i$), which naturally guides the solver towards integer solutions early in the search tree. This architectural shift enables GLPK to effortlessly solve dense 9-target problems to integer optimality at the root node in under **0.15 seconds**â€”achieving a computational speedup of over 10,000x compared to the previous model.

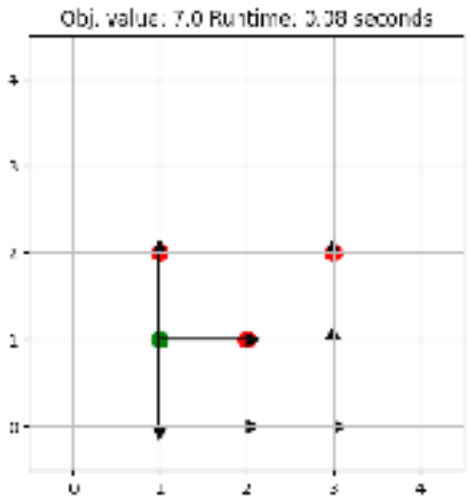


Figure 7. Static terminal nodes.

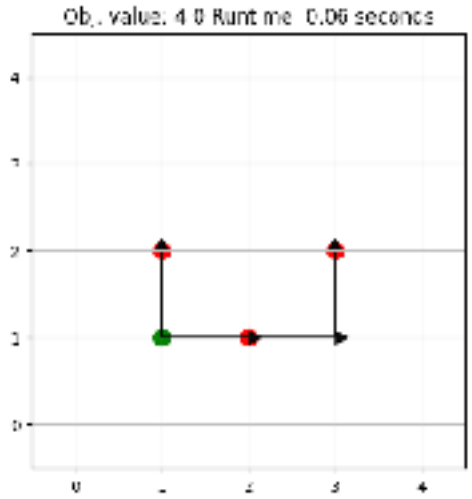


Figure 8. Flexible terminal nodes.

Extended Results

The multi-commodity formulation was further tested on a symmetrical hexagonal grid to evaluate its performance across dynamically scaled target distributions. The target nodes were systematically modified (moved closer, extended outward, and shifted laterally) to form the 3-Target, 6-Target, and 9-Target problem instances.

As demonstrated in the subsequent figures, the algorithm flawlessly identifies the optimal pass-through paths, confirming that the revised flow conservation constraints allow targets to be flexibly traversed without restricting them to terminal leaves. Even in the dense 9-Target scenario, the tight LP relaxation allows the solver to reach integer optimality in a fraction of a second.

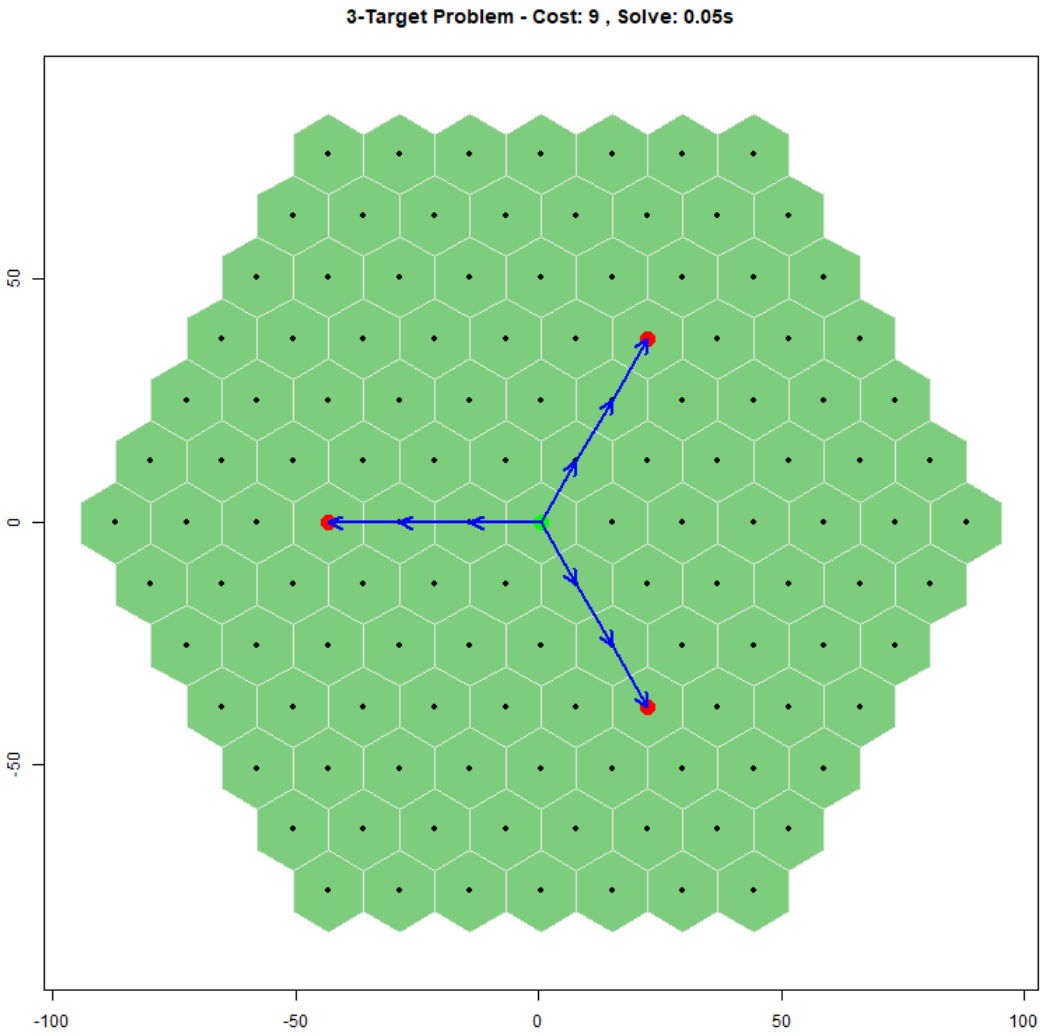


Figure 9. 3-Target Problem.

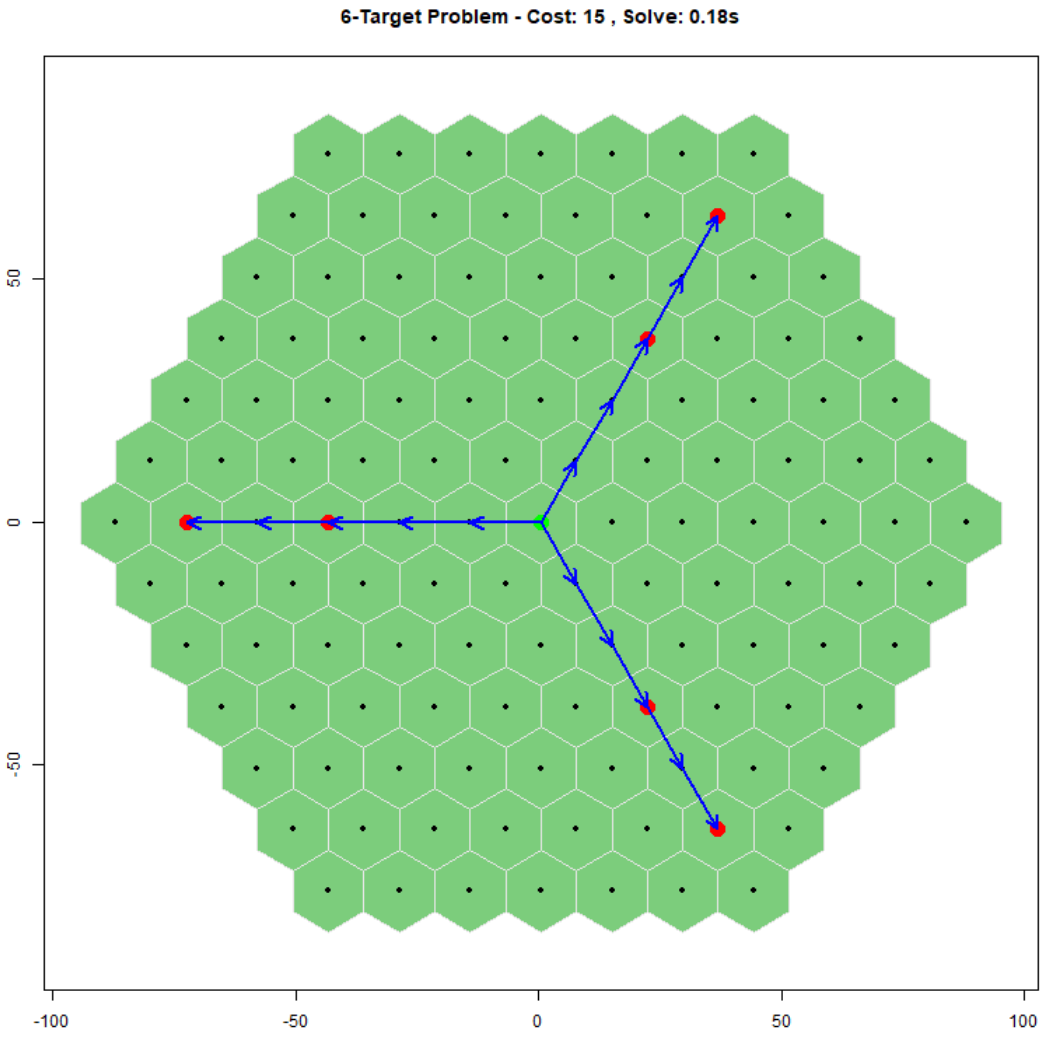


Figure 10. 6-Target Problem.

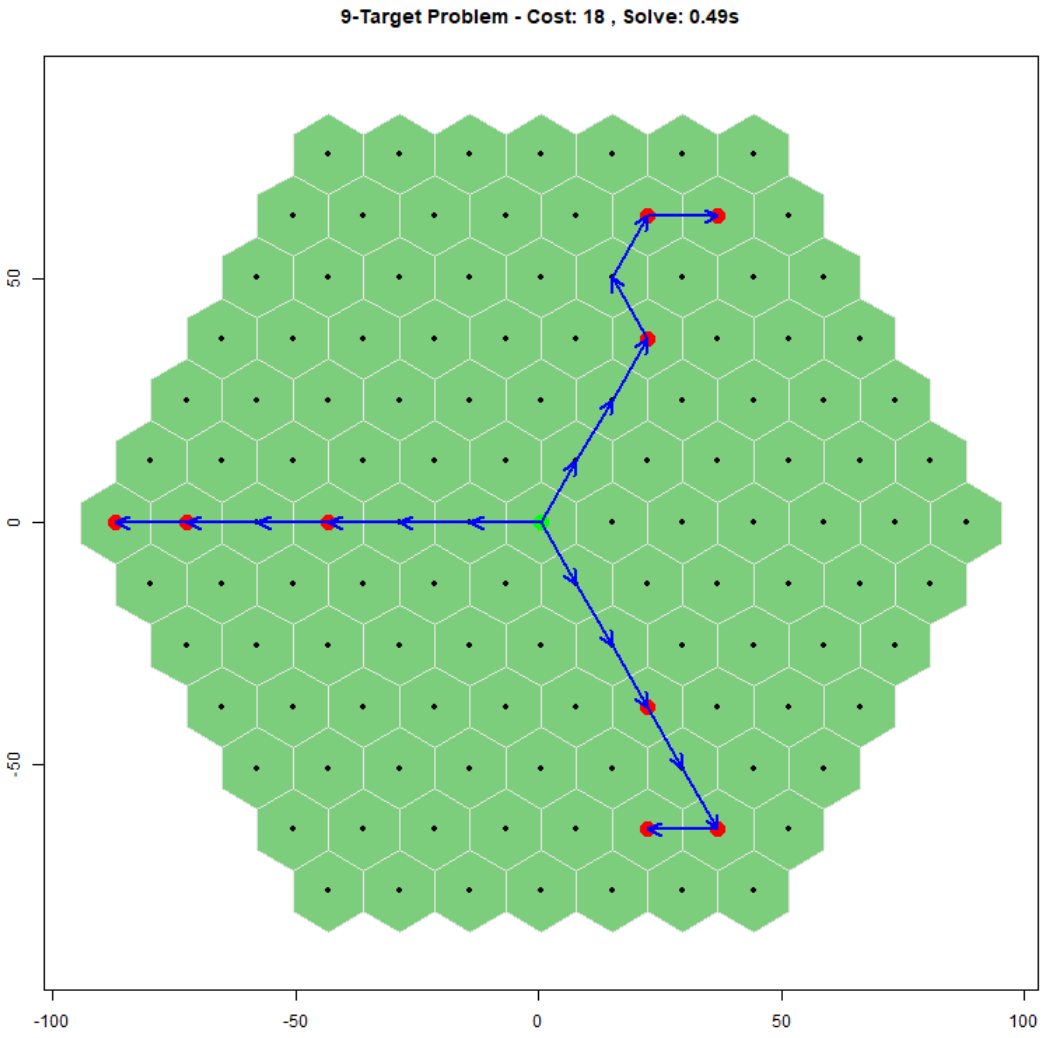


Figure 11. 9-Target Problem.